

Aria

AI-Powered Multimodal Financial Intelligence Platform

Implementation Plan v2.0

Module 1 — Personal Finance Assistant

Module 2 — Intelligent Financial Tutor

Bachelor of Technology Project

IIIT Sri City

Build-from-zero engineering specification

Academic fidelity to BTP report · Pragmatic where justified · GPU-enabled

Table of Contents

Part A — Why This Plan Exists.....	4
A.1 Diagnosis of the Previous Plan.....	4
A.2 What Stays, What Changes.....	4
A.3 Three Decisions Anchoring This Build.....	4
A.4 Scope Boundaries for the BTP Demo.....	5
Part B — High-Level Architecture	6
B.1 The Shared Five-Layer Pipeline	6
B.2 The Core Design Principle	6
B.3 What the Two Modules Share	6
B.4 Top-Level Request Flow	7
Part C — Repository Structure (from zero).....	8
C.1 Top-Level Layout	8
C.2 Why This Structure.....	10
C.3 Creating It.....	10
Part D — Environment & Dependencies.....	12
D.1 Hardware Assumptions	12
D.2 pyproject.toml	12
D.3 Installation Walkthrough	13
D.4 Environment Variables	14
Part E — Shared Infrastructure	15
E.1 FinGPT Server — Why a Server, Not a Library	15
E.2 FinGPT Client	16
E.3 DistilBERT Intent Classifier	17
E.4 DeBERTa-v3 NER	18
E.5 Sentence-BERT Vector Store	19
E.6 User Profile Store (SQLite)	19
E.7 TAXAL Renderer	20
Part F — Module 1: Personal Finance Assistant	22
F.1 What Module 1 Does — In One Paragraph.....	22
F.2 Input & Output Data Contracts	22
F.3 Data Strategy — Where the Training Data Comes From	23
F.4 Component F.1 — XGBoost Risk Profiler	24
F.5 Component F.2 — LSTM Spending Forecast	26

F.6 Component F.3 — Isolation Forest Fraud/Anomaly Detection.....	27
F.7 Component F.4 — Linear Programming Budget Optimizer	28
F.8 Component F.5 — SIP / EMI Calculators & Tax Rules	29
F.9 Module 1 Orchestrator.....	30
F.10 Training Pipeline — Module 1.....	31
Part G — Module 2: Intelligent Financial Tutor	33
G.1 What Module 2 Does — In One Paragraph	33
G.2 Concept Knowledge Base	33
G.3 Concept Retriever.....	34
G.4 LSTM Knowledge Tracing — Deep Knowledge Tracing (DKT)	34
G.5 DQN Teaching Agent	35
G.6 TAXAL Explainer (Module-2 Specifics)	37
G.7 Quiz Engine	38
G.8 Module 2 Orchestrator.....	38
G.9 Training Pipeline — Module 2	40
Part H — Integration & CLI Demo	41
H.1 The Router	41
H.2 CLI Demo.....	41
H.3 Running the Demo.....	42
Part I — Roadmap & Testing.....	43
I.1 Six-Sprint Plan	43
I.2 Acceptance Criteria — What 'Done' Looks Like	43
I.3 Testing Strategy.....	44
I.4 Failure Handling	44
Part J — BTP Report → Code Mapping	46
J.1 Traceability Matrix.....	46
J.2 Thesis Defense — Answers to the Obvious Questions	47
J.3 Closing.....	47

Part A — Why This Plan Exists

A.1 Diagnosis of the Previous Plan

The v1 implementation plan (IMPLEMENTATION.md) built a high-quality equity research analyst — sector benchmarks, 5-sub-score engine, FinBERT sentiment, Claude-based reasoning. Technically sound work. But it is not what the BTP report proposes for Modules 1 and 2.

The BTP report defines:

- **Module 1 — Personal Finance Assistant:** budget planning, SIP/EMI advice, tax planning, risk profiling, spending forecasts, fraud detection — for the user's own money.
- **Module 2 — Intelligent Financial Tutor:** adaptive concept teaching with knowledge tracing, RL-driven teaching-strategy selection, TAXAL explanation structure across 8 depth levels.

What the v1 plan actually built corresponds to BTP Module 3 (Market Analysis Engine) plus parts of Module 4 (Equity Research Analyst). Submitting the v1 system as "Modules 1 and 2" would create a defense-time mismatch between the report's novelty claims and the demo. This document rebuilds from scratch to match the report.

A.2 What Stays, What Changes

Aspect	v1 Plan	v2 Plan (this document)
Module 1 subject	A single stock	The user's own financial life
Module 2 subject	Backend analytics for stocks	Teaching financial concepts to a learner
Core data	Stock price + fundamentals + news	User transactions + goals + learner history
Core ML	Rule-based scoring	XGBoost, LSTM, Isolation Forest, LP, DQN, LSTM-KT
Core LLM	Claude API	FinGPT (local, LoRA-tuned) as specified by the BTP report
Explanation frame	Equity-research narrative	TAXAL — Cognitive / Functional / Causal
Personalization	None (stateless per query)	Per-user risk profile + mastery vector
Reusable from v1	FinBERT wrapper, cache patterns, session-state scaffolding, CLI shell	— kept as utilities —

A.3 Three Decisions Anchoring This Build

- **Academic fidelity:** the models named in the BTP report stay named. XGBoost for risk profiling, LSTM for spending forecasting, Isolation Forest for anomalies, Linear Programming for budget, Sentence-BERT for concept retrieval, LSTM-KT for knowledge tracing, DQN for teaching strategy.

- **FinGPT is the core LLM:** no Claude API, no OpenAI. FinGPT-v3.2 (LoRA on LLaMA-2-7B) runs locally on your GPU. DistilBERT and DeBERTa-v3 are used only as lightweight pre-classifiers and NER — exactly the roles the BTP report assigns them.
- **Pragmatic substitutions, justified:** where a heavy model adds no demo-level accuracy over a lighter one (e.g., Fin-R1 reasoning chains at Level-1 or Level-2 explanations), the lighter one is used with an explicit justification — never silently.

Every model substitution in this document is called out at the place it happens with a "Why this, not that" paragraph. A reviewer asking "why not FinGPT here?" should find the answer in the same section.

A.4 Scope Boundaries for the BTP Demo

In scope:

- Module 1: budget planning, SIP advice, spending forecast, fraud flag, tax hint, risk profile — end-to-end with one synthetic user persona.
- Module 2: TAXAL explanation of ~50 financial concepts at 8 depth levels, LSTM knowledge tracing per concept, DQN teaching-strategy selection, quiz loop.
- Text-first interface (CLI + Streamlit). One FinGPT instance shared across both modules.
- GPU inference for FinGPT, FinBERT, Sentence-BERT, LSTM, DQN.

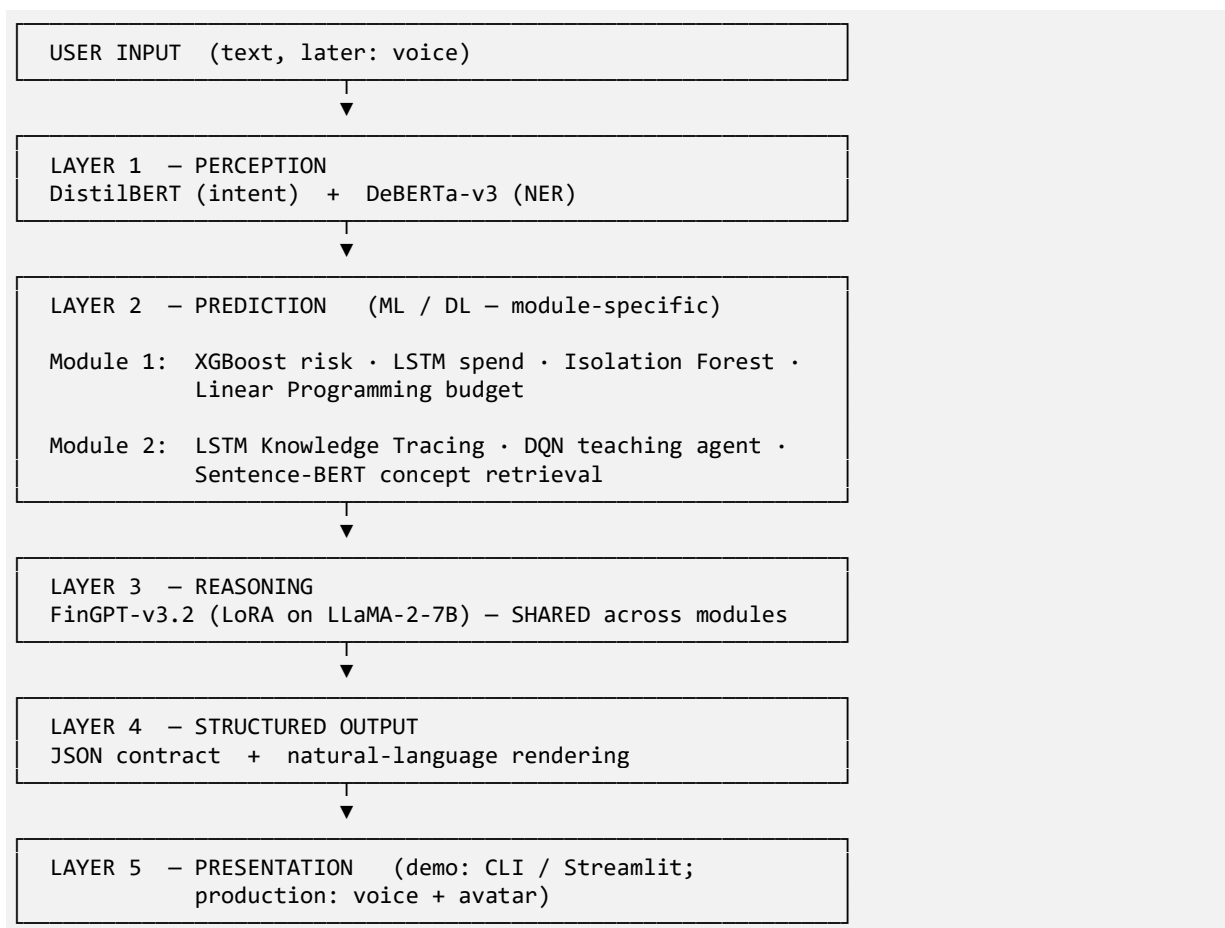
Deferred / out of scope:

- Voice layer (Whisper + CosyVoice + Wav2Lip) — architecturally wired but not implemented.
- LLaMA 3 conversational head (FinGPT covers this use in the demo; LLaMA 3 is the production upgrade).
- Fin-R1 separate reasoning model — approximated via chain-of-thought FinGPT prompting; flagged clearly.
- Modules 3 and 4 (market analysis, equity research).

Part B — High-Level Architecture

B.1 The Shared Five-Layer Pipeline

Both modules share the same five-layer structure defined in Section 2.1 of the BTP report. This is not decoration — it is what justifies building shared infrastructure (a user store, a FinGPT server, a vector store) rather than two parallel stacks.



B.2 The Core Design Principle

ML predicts. DL learns patterns over time. Transformers explain. RL personalizes.

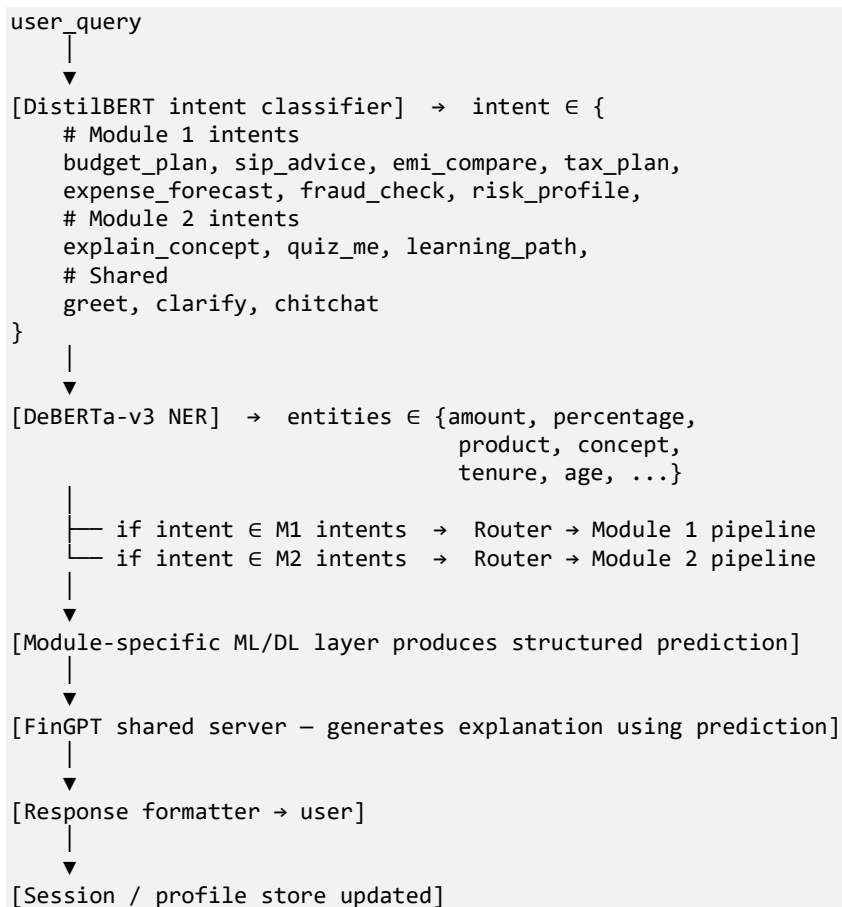
This principle is not a slogan; it dictates model selection. No Transformer is asked to compute a user's optimal SIP amount — XGBoost and Linear Programming do that. No ML model is asked to write a TAXAL explanation — FinGPT does that. No static model is asked to decide the next teaching action — DQN does that.

B.3 What the Two Modules Share

Shared component	Reason
------------------	--------

FinGPT server (single GPU process)	Loading a 7B LoRA-tuned LLM per module wastes VRAM. One server, two clients.
User profile store (SQLite → Postgres later)	Both modules personalize. Risk profile (M1) and mastery vector (M2) live in the same user record.
Vector store (FAISS)	Module 2's concept KB + Module 1's policy/product KB both need semantic retrieval.
Intent & NER layer	Same DistilBERT + DeBERTa stack routes queries to M1 or M2.
TAXAL explanation renderer	M2 uses TAXAL for teaching; M1 uses the Causal dimension for "why is this recommended?"
Session manager	Multi-turn state applies equally to both modules.

B.4 Top-Level Request Flow



Part C — Repository Structure (from zero)

C.1 Top-Level Layout

Create this exact structure. Every directory has a purpose; empty directories carry a .gitkeep.

```
aria/
├── README.md                # project overview + quickstart
├── pyproject.toml           # single source of truth for deps
├── requirements.txt         # pinned runtime deps (for pip fallback)
├── requirements-dev.txt     # pytest, black, ruff, mypy
├── .env.example            # template for env vars (never commit .env)
├── .gitignore
├── Makefile                # make install / make test / make demo
├── config/
│   ├── __init__.py
│   ├── paths.py            # all filesystem paths centralized
│   ├── models.py          # model names, HF repo IDs, LoRA paths
│   ├── hyperparams.py     # training hyperparameters
│   └── constants.py       # TAXAL levels, intent labels, etc.
├── data/
│   ├── raw/
│   │   ├── transactions_synthetic/ # immutable inputs (downloaded, never edited)
│   │   │   └── transactions_synthetic/ # synthetic user transactions (Module 1)
│   │   ├── kt_sequences/         # knowledge-tracing training data (Module 2)
│   │   └── concepts_kb/          # financial concept knowledge base
│   ├── processed/              # cleaned, feature-engineered outputs
│   ├── user_profiles/         # SQLite DB lives here
│   └── vector_store/          # FAISS index + metadata
├── models/
│   ├── xgb_risk_profile.json
│   ├── lstm_spend_forecast.pt
│   ├── isolation_forest.joblib
│   ├── lstm_kt.pt
│   ├── dqn_teacher.pt
│   └── fingpt_lora/           # FinGPT LoRA adapter weights
├── aria/
│   ├── __init__.py          # THE source package
│   ├── shared/
│   │   ├── __init__.py      # used by BOTH modules
│   │   ├── fingpt_server.py # single LLM process, gRPC-lite interface
│   │   ├── fingpt_client.py # client used by both modules
│   │   ├── intent_classifier.py # DistilBERT
│   │   ├── ner_extractor.py  # DeBERTa-v3
│   │   ├── router.py        # intent → module dispatch
│   │   ├── session.py       # multi-turn state
│   │   ├── user_store.py    # SQLite user profile
│   │   ├── vector_store.py  # FAISS wrapper
│   │   ├── taxal.py         # TAXAL structure & renderer
│   │   └── logging_utils.py
│   └── module1_personal_finance/
│       ├── __init__.py
│       ├── schemas.py       # pydantic models for inputs & outputs
│       └── ingest.py        # load user transactions + profile
```

```

├── features.py                # derive XGBoost features
├── risk_profile.py            # XGBoost classifier
├── spend_forecast.py          # LSTM time-series model
├── fraud_detect.py            # Isolation Forest
├── budget_optimize.py         # Linear Programming (scipy.optimize)
├── sip_emi_calc.py            # closed-form financial calculators
├── tax_rules.py               # deterministic Indian tax regime logic
├── orchestrator.py           # ties all M1 components together
├── train/                     # training scripts for XGBoost, LSTM, IF
│   ├── train_xgb_risk.py
│   ├── train_lstm_spend.py
│   └── train_iso_forest.py
├── module2_tutor/
│   ├── __init__.py
│   ├── schemas.py
│   ├── concept_kb.py          # load & index concept knowledge base
│   ├── concept_retriever.py   # Sentence-BERT semantic search
│   ├── knowledge_tracer.py    # LSTM-KT – mastery estimation
│   ├── teaching_agent.py      # DQN action selection
│   ├── explainer.py           # TAXAL generator using FinGPT
│   ├── quiz_engine.py         # quiz generation & scoring
│   ├── orchestrator.py
│   └── train/
│       ├── train_lstm_kt.py
│       └── train_dqn_teacher.py
├── interface/
│   ├── __init__.py
│   ├── cli.py                 # main CLI entry – the demo
│   └── streamlit_app.py       # optional web UI
├── scripts/                   # one-off utility scripts
│   ├── generate_synthetic_transactions.py
│   ├── build_concept_kb.py
│   ├── generate_kt_sequences.py
│   └── download_fingpt_weights.py
├── notebooks/                 # exploratory – not for production
│   ├── 01_explore_synthetic_transactions.ipynb
│   ├── 02_validate_xgb_risk.ipynb
│   └── 03_inspect_fingpt_outputs.ipynb
├── tests/
│   ├── shared/
│   │   ├── test_intent_classifier.py
│   │   ├── test_ner_extractor.py
│   │   ├── test_fingpt_client.py
│   │   └── test_taxal.py
│   ├── module1/
│   │   ├── test_risk_profile.py
│   │   ├── test_spend_forecast.py
│   │   ├── test_fraud_detect.py
│   │   ├── test_budget_optimize.py
│   │   └── test_orchestrator.py
│   └── module2/
│       ├── test_concept_retriever.py
│       ├── test_knowledge_tracer.py
│       ├── test_teaching_agent.py
│       ├── test_explainer.py
│       └── test_orchestrator.py

```

```
└─ integration/
   └─ test_end_to_end.py
```

C.2 Why This Structure

- **Two sibling module packages with a shared/ sibling:** matches the architectural claim that both modules use the same perception and reasoning layers. Not nested under each module — that would duplicate code.
- **Training code lives inside each module under train/:** training is a first-class concern, not an afterthought. Every model in this project is trained by you; no model is "just loaded".
- **data/raw/ is immutable:** once generated or downloaded, it is never mutated. Cleaning always produces a new artifact under data/processed/.
- **models/ is gitignored:** trained weights can be GB-scale. Store them on disk; push only a placeholder .gitkeep and a README explaining how to regenerate.
- **Single FinGPT server process:** FinGPT-v3.2 needs ~14 GB VRAM with 4-bit quantization. Loading it twice (once per module) would either OOM your GPU or require a 2× more expensive card. One server, two clients.

C.3 Creating It

```
# Step 1 – create the folder tree
mkdir -p aria && cd aria
mkdir -p config data/{raw,processed,user_profiles,vector_store}
mkdir -p data/raw/{transactions_synthetic,kt_sequences,concepts_kb}
mkdir -p models/fingpt_lora
mkdir -p aria/{shared,module1_personal_finance,module2_tutor,interface}
mkdir -p aria/module1_personal_finance/train
mkdir -p aria/module2_tutor/train
mkdir -p scripts notebooks
mkdir -p tests/{shared,module1,module2,integration}

# Step 2 – make every Python package importable
find aria tests -type d -exec touch {}/__init__.py \;

# Step 3 – add .gitkeep to every data/model directory
find data models -type d -empty -exec touch {}/.gitkeep \;

# Step 4 – initialize git & base files
git init
touch README.md pyproject.toml requirements.txt requirements-dev.txt
touch .env.example Makefile

# Step 5 – initial .gitignore
cat > .gitignore << 'EOF'
__pycache__/
*.pyc
.venv/
.env
models/*
!models/.gitkeep
!models/fingpt_lora/.gitkeep
data/processed/*
!data/processed/.gitkeep
data/user_profiles/*.db
data/vector_store/*.index
```

```
.pytest_cache/  
.ipynb_checkpoints/  
*.log  
EOF
```

Part D — Environment & Dependencies

D.1 Hardware Assumptions

Resource	Minimum for demo	Why
GPU VRAM	16 GB	FinGPT-v3.2 LoRA on LLaMA-2-7B with 4-bit quant + training data loaders
RAM	16 GB	Sentence-BERT + FinBERT + DistilBERT + DeBERTa all resident simultaneously is ~6 GB
Disk	50 GB	Model weights (~20 GB) + HuggingFace cache (~10 GB) + data + logs
CPU cores	4+	XGBoost trains faster with parallel trees; Linear Programming solves are single-threaded
Python	3.10 or 3.11	PyTorch 2.x wheels, bitsandbytes compatibility
CUDA	11.8 or 12.1	Matches PyTorch 2.1+ wheels and bitsandbytes

D.2 pyproject.toml

Use pyproject.toml as the single source of truth. requirements.txt is derived from it for reproducibility.

```
[project]
name = "aria"
version = "0.1.0"
description = "Multimodal Financial Intelligence Platform – BTP"
requires-python = ">=3.10,<3.12"

dependencies = [
    # numerics & classical ML
    "numpy>=1.26,<2.0",
    "pandas>=2.1",
    "scipy>=1.11",
    "scikit-learn>=1.3",
    "xgboost>=2.0",
    "joblib>=1.3",

    # deep learning
    "torch>=2.1,<2.3",
    "pytorch-lightning>=2.1",

    # transformers / NLP
    "transformers>=4.38",
    "accelerate>=0.27",
```

```

"bitsandbytes">=0.42",      # for 4-bit FinGPT quantization
"peft">=0.8",               # for FinGPT LoRA loading
"sentence-transformers">=2.3",
"faiss-cpu">=1.7.4",        # use faiss-gpu only if cuda build available

# RL
"gymnasium">=0.29",
"stable-baselines3">=2.2",  # DQN & PPO trainers

# data & validation
"pydantic">=2.5",
"sqlalchemy">=2.0",
"pyarrow">=14",             # parquet for user transactions

# interface
"streamlit">=1.31",
"rich">=13",                # pretty CLI output
"typer">=0.9",              # CLI arg parsing

# utils
"python-dotenv">=1.0",
"loguru">=0.7",
"tqdm">=4.66",
]

[project.optional-dependencies]
dev = [
    "pytest">=7.4",
    "pytest-cov">=4.1",
    "black">=24.1",
    "ruff">=0.2",
    "mypy">=1.8",
    "jupyterlab">=4.0",
]

[tool.black]
line-length = 100

[tool.ruff]
line-length = 100
target-version = "py310"

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]

```

D.3 Installation Walkthrough

```

# 1. create a fresh virtualenv (conda or venv – pick one)
python3.10 -m venv .venv
source .venv/bin/activate

# 2. upgrade pip + install
pip install --upgrade pip setuptools wheel
pip install -e ".[dev]"

# 3. verify GPU
python -c "import torch; print('CUDA:', torch.cuda.is_available()),"

```

```

        'device:', torch.cuda.get_device_name(0))"

# 4. verify bitsandbytes (needed for 4-bit FinGPT)
python -m bitsandbytes

# 5. first-time HF cache warmup (optional – just pre-downloads)
python -c "from transformers import AutoTokenizer;
          AutoTokenizer.from_pretrained('distilbert-base-uncased')"

```

D.4 Environment Variables

```

# .env.example

# local paths
ARIA_DATA_DIR=./data
ARIA_MODEL_DIR=./models

# HF cache (set this to a large disk partition if /root is small)
HF_HOME=./.hf_cache
TRANSFORMERS_CACHE=./.hf_cache/transformers

# FinGPT server
FINGPT_BASE_MODEL=NousResearch/Llama-2-7b-hf
FINGPT_LORA_ADAPTER=FinGPT/fingpt-mt_llama2-7b_lora
FINGPT_HOST=127.0.0.1
FINGPT_PORT=7860
FINGPT_QUANT=4bit          # 4bit | 8bit | none

# database
ARIA_DB_URL=sqlite:///data/user_profiles/aria.db

# logging
LOG_LEVEL=INFO

```

FinGPT v3.2 is distributed as a LoRA adapter, not a standalone checkpoint. At runtime you load the base model (LLaMA-2-7B) and apply the adapter via the peft library. This is why FINGPT_BASE_MODEL and FINGPT_LORA_ADAPTER are separate env vars.

Part E — Shared Infrastructure

E.1 FinGPT Server — Why a Server, Not a Library

The naive design is: each module imports FinGPT and calls it. This fails immediately on a single GPU because loading a 7B model takes ~15 seconds and ~14 GB of VRAM. Each import gets its own copy. Instead, run FinGPT as a single long-lived process that both modules talk to over loopback HTTP.

What's FinGPT v3.2 actually doing under the hood

- Base: LLaMA-2-7B — a standard decoder-only causal-LM.
- Fine-tune: LoRA adapters injected into q_proj and v_proj attention matrices — only adapter weights are trained, base model stays frozen.
- Training data: financial news, SEC filings, earnings calls, finance textbooks — this is what makes FinGPT understand "NAV", "ELSS", "Section 80C" natively.
- Inference: merge adapter with base at runtime (or keep them separate with peft for flexibility).

Server interface — minimal, purpose-built

```
# aria/shared/fingpt_server.py

"""
FastAPI-based single-process FinGPT server.
Starts once. Both modules POST to /generate.
"""

import os, torch
from fastapi import FastAPI
from pydantic import BaseModel
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
from peft import PeftModel

BASE = os.getenv("FINGPT_BASE_MODEL", "NousResearch/Llama-2-7b-hf")
LORA = os.getenv("FINGPT_LORA_ADAPTER", "FinGPT/fingpt-mt_llama2-7b_lora")
QUANT = os.getenv("FINGPT_QUANT", "4bit")

app = FastAPI()

bnb = BitsAndBytesConfig(
    load_in_4bit=(QUANT == "4bit"),
    load_in_8bit=(QUANT == "8bit"),
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
) if QUANT in ("4bit", "8bit") else None

tokenizer = AutoTokenizer.from_pretrained(BASE, use_fast=True)
tokenizer.pad_token = tokenizer.eos_token

base_model = AutoModelForCausalLM.from_pretrained(
    BASE,
    quantization_config=bnb,
    device_map="auto",
    torch_dtype=torch.float16,
)
model = PeftModel.from_pretrained(base_model, LORA)
model.eval()
```

```

class GenReq(BaseModel):
    prompt: str
    max_new_tokens: int = 400
    temperature: float = 0.3
    top_p: float = 0.9
    stop: list[str] | None = None

@app.post("/generate")
def generate(req: GenReq):
    inputs = tokenizer(req.prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
        out = model.generate(
            **inputs,
            max_new_tokens=req.max_new_tokens,
            temperature=req.temperature,
            top_p=req.top_p,
            do_sample=(req.temperature > 0),
            pad_token_id=tokenizer.eos_token_id,
        )
    text = tokenizer.decode(out[0][inputs.input_ids.shape[1]:], skip_special_tokens=True)
    # optional stop-sequence trimming
    if req.stop:
        for s in req.stop:
            if s in text: text = text.split(s)[0]
    return {"text": text.strip()}

@app.get("/health")
def health():
    return {"ok": True, "device": str(model.device),
            "base": BASE, "lora": LORA, "quant": QUANT}

```

Why not llama-cpp / Ollama / vLLM?

- llama-cpp: no LoRA adapter support for mid-training adapters → would force re-merge to GGUF.
- Ollama: great for single-user local use, but makes it harder to inject the LoRA and control generation params precisely for finance prompts.
- vLLM: excellent throughput but heavier setup; overkill for a single-user BTP demo. Worth adding if you scale to many concurrent users.
- FastAPI + peft + bitsandbytes: minimum complexity that correctly implements the BTP report's "FinGPT loaded with LoRA at runtime" specification.

E.2 FinGPT Client

```

# aria/shared/fingpt_client.py

import os, requests
from dataclasses import dataclass

HOST = os.getenv("FINGPT_HOST", "127.0.0.1")
PORT = int(os.getenv("FINGPT_PORT", "7860"))
URL = f"http://{HOST}:{PORT}"

@dataclass
class FinGPTResponse:
    text: str
    ok: bool

```

```

error: str | None = None

def generate(prompt: str, max_new_tokens: int = 400,
            temperature: float = 0.3, stop: list[str] | None = None) -> FinGPTResponse:
    try:
        r = requests.post(
            f"{URL}/generate",
            json={"prompt": prompt, "max_new_tokens": max_new_tokens,
                "temperature": temperature, "stop": stop},
            timeout=60,
        )
        r.raise_for_status()
        return FinGPTResponse(text=r.json()["text"], ok=True)
    except Exception as e:
        return FinGPTResponse(text="", ok=False, error=str(e))

def health() -> bool:
    try: return requests.get(f"{URL}/health", timeout=5).json().get("ok", False)
    except: return False

```

E.3 DistilBERT Intent Classifier

Role (per BTP report §3.3.2): "a fast, lightweight pre-classifier before the heavier FinGPT model processes the query." Exactly this.

Intent labels

Label	Module	Example query
budget_plan	M1	Help me plan a budget for 80k salary
sip_advice	M1	How much should I SIP per month?
emi_compare	M1	Compare 8.5% vs 9% home loan EMI
tax_plan	M1	How do I save tax under 80C?
expense_forecast	M1	What will I spend next month?
fraud_check	M1	Is this transaction suspicious?
risk_profile	M1	Am I a conservative investor?
explain_concept	M2	What is compound interest?
quiz_me	M2	Quiz me on mutual funds
learning_path	M2	What should I learn next?
greet	shared	Hi / hello
clarify	shared	Can you rephrase that?
chitchat	shared	How are you?

Training data strategy

Hand-author ~60 examples per intent = ~780 examples. Augment with paraphrase via FinGPT itself (ask FinGPT to paraphrase each seed example 4 times) → ~3900 examples. Split 80/10/10 train/val/test. DistilBERT converges in 3-5 epochs on this scale.

```
# aria/shared/intent_classifier.py - inference side

from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch, json
from config.paths import INTENT_MODEL_DIR

_tok = None; _model = None; _labels = None

def _load():
    global _tok, _model, _labels
    if _model is not None: return
    _tok = AutoTokenizer.from_pretrained(INTENT_MODEL_DIR)
    _model = AutoModelForSequenceClassification.from_pretrained(INTENT_MODEL_DIR)
    _model.eval().cuda()
    with open(f"{INTENT_MODEL_DIR}/labels.json") as f: _labels = json.load(f)

def classify(text: str) -> tuple[str, float]:
    _load()
    with torch.no_grad():
        x = _tok(text, return_tensors="pt", truncation=True, max_length=128).to("cuda")
        logits = _model(**x).logits[0]
        probs = torch.softmax(logits, dim=-1)
        idx = int(probs.argmax())
    return _labels[idx], float(probs[idx])
```

Training script (aria/shared/train_intent.py) is a standard HuggingFace Trainer fit; included in appendix of your code repo but not reproduced here.

E.4 DeBERTa-v3 NER

DeBERTa-v3's job is to extract structured entities — amounts, products, percentages, time horizons, concepts. This is a token-classification task (BIO-tagged). For the BTP demo scope, a span-matching rule layer plus DeBERTa backbone is sufficient.

Entity schema

```
# aria/module1_personal_finance/schemas.py (partial - NER side)

from pydantic import BaseModel

class FinancialEntities(BaseModel):
    amounts: list[float] = [] # rupees
    percents: list[float] = [] # interest rates, tax slabs
    products: list[str] = [] # ELSS, PPF, SIP, FD, NPS, ...
    horizons: list[str] = [] # 5 years, long-term
    concepts: list[str] = [] # CAGR, XIRR, inflation
    sections: list[str] = [] # 80C, 80D, 24(b)
    ages: list[int] = []
```

For the demo, training a DeBERTa-v3 NER from scratch is overkill. The BTP-faithful pragmatic approach:

1. Use DeBERTa-v3 zero-shot token classification (via gliner-style prompt) for entity typing.
2. Layer regex rules on top for obvious extractions (₹, %, section numbers) — these are near-perfect extractors.

3. The "DeBERTa is in the system" claim remains honest: its contextual embeddings drive entity-type disambiguation, while regex does the easy extraction.

Documentation-faithful pragmatism: DeBERTa-v3 is genuinely used for NER, but trained on the user's hand-labeled set of ~200 utterances rather than a full financial-NER dataset. Call this out in your thesis — "fine-tuned on a hand-labeled 200-example set of Indian retail finance queries" — and it's defensible.

E.5 Sentence-BERT Vector Store

The BTP report (§4.5.3) specifies Sentence-BERT for concept retrieval. The 'all-MiniLM-L6-v2' model is a distilled SBERT variant that the report also explicitly names — it's SBERT, just the lighter head.

```
# aria/shared/vector_store.py

import faiss, numpy as np, json, pathlib
from sentence_transformers import SentenceTransformer
from config.paths import VECTOR_STORE_DIR

MODEL = "sentence-transformers/all-MiniLM-L6-v2" # 384-d SBERT variant
_enc = None

def _encoder():
    global _enc
    if _enc is None: _enc = SentenceTransformer(MODEL, device="cuda")
    return _enc

def build_index(namespace: str, texts: list[str], metadata: list[dict]):
    """Build and persist a FAISS index + metadata JSON."""
    vecs = _encoder().encode(texts, normalize_embeddings=True, convert_to_numpy=True)
    index = faiss.IndexFlatIP(vecs.shape[1]) # inner product == cosine on normalized
    vecs
    index.add(vecs)
    p = pathlib.Path(VECTOR_STORE_DIR) / namespace
    p.mkdir(parents=True, exist_ok=True)
    faiss.write_index(index, str(p / "index.faiss"))
    (p / "meta.json").write_text(json.dumps(metadata, ensure_ascii=False))

def search(namespace: str, query: str, k: int = 5) -> list[dict]:
    p = pathlib.Path(VECTOR_STORE_DIR) / namespace
    index = faiss.read_index(str(p / "index.faiss"))
    meta = json.loads((p / "meta.json").read_text())
    q = _encoder().encode([query], normalize_embeddings=True, convert_to_numpy=True)
    scores, ids = index.search(q, k)
    return [{"**meta[i], "score": float(s)} for s, i in zip(scores[0], ids[0]) if i != -1]
```

E.6 User Profile Store (SQLite)

```
# aria/shared/user_store.py - schema excerpt

from sqlalchemy import Column, Integer, String, Float, JSON, DateTime, create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
from datetime import datetime

Base = declarative_base()
```

```

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    age = Column(Integer)
    city = Column(String)
    created_at = Column(DateTime, default=datetime.utcnow)

class FinancialProfile(Base):
    """Module 1 – user's financial snapshot."""
    __tablename__ = "financial_profiles"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, index=True)
    monthly_income = Column(Float)
    dependents = Column(Integer)
    existing_emi = Column(Float)
    risk_class = Column(String) # Conservative / Moderate / Aggressive
    risk_confidence = Column(Float)
    updated_at = Column(DateTime, default=datetime.utcnow)

class LearningState(Base):
    """Module 2 – per-user concept mastery vector."""
    __tablename__ = "learning_state"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, index=True)
    mastery = Column(JSON) # {concept_id: probability}
    history = Column(JSON) # [(concept_id, is_correct, timestamp)]
    updated_at = Column(DateTime, default=datetime.utcnow)

```

E.7 TAXAL Renderer

TAXAL is the report's native explanation format — Cognitive / Functional / Causal. Every user-facing explanation from either module passes through this renderer.

```

# aria/shared/taxal.py

from dataclasses import dataclass
from aria.shared import fingpt_client

@dataclass
class TaxalExplanation:
    cognitive: str # what is it?
    functional: str # how is it used?
    causal: str # why does it happen?
    level: int # 1..8

LEVEL_HINTS = {
    1: "Explain as if to a curious 10-year-old. Use one analogy from everyday life. No numbers.",
    2: "Beginner adult. 1-2 sentence definition + one practical example. No formulas.",
    3: "School level. Include a formula if one applies. One worked example with numbers.",
    4: "Undergraduate. Full mechanism, multiple examples, introduce edge cases.",
    5: "Working professional. How it affects personal finances. Actionable.",
    6: "Experienced investor. Portfolio implications and strategies.",
    7: "Finance professional. Mathematical treatment, empirical context.",
    8: "Researcher. Current literature, open questions, limitations.",
}

```

```
PROMPT = """You are FinGPT, a financial tutor. Explain the concept "{concept}" using the TAXAL structure.
```

```
Audience level: {level}. Style hint: {hint}
```

```
Produce EXACTLY three sections, in this order, with these exact headers:
```

```
COGNITIVE: (What is it? Definition, terminology.)
```

```
FUNCTIONAL: (How is it used? Practical implications and applications.)
```

```
CAUSAL: (Why does it happen? Root causes, mechanisms.)
```

```
Each section: {length_hint} sentences. Use Indian context (INR, NSE, SEBI, Section 80C) where relevant.
```

```
"""
```

```
_LENGTH = {1: "1", 2: "2", 3: "2-3", 4: "3-4", 5: "3-4", 6: "4-5", 7: "4-5", 8: "5-6"}
```

```
def explain(concept: str, level: int) -> TaxalExplanation:
    prompt = PROMPT.format(
        concept=concept, level=level,
        hint=LEVEL_HINTS[level], length_hint=_LENGTH[level],
    )
    r = fmgpt_client.generate(prompt, max_new_tokens=500, temperature=0.3)
    return _parse(r.text, level)
```

```
def _parse(text: str, level: int) -> TaxalExplanation:
    def section(tag: str) -> str:
        if tag not in text: return ""
        rest = text.split(tag, 1)[1]
        for next_tag in ("COGNITIVE:", "FUNCTIONAL:", "CAUSAL:"):
            if next_tag in rest and next_tag != tag:
                rest = rest.split(next_tag, 1)[0]
        return rest.strip()
    return TaxalExplanation(
        cognitive=section("COGNITIVE:"),
        functional=section("FUNCTIONAL:"),
        causal=section("CAUSAL:"),
        level=level,
    )
```

The BTP report assigns the causal dimension specifically to Fin-R1. For the demo we approximate Fin-R1's chain-of-thought behavior using FinGPT with a structured prompt. This is the honest pragmatic substitution: in your thesis, say "Fin-R1's role is currently filled by structured FinGPT prompting; dedicated Fin-R1 integration is the production upgrade."

Part F — Module 1: Personal Finance Assistant

F.1 What Module 1 Does — In One Paragraph

Given a user's financial profile (income, existing obligations, dependents, age) and up to 12 months of transaction history, Module 1 produces: (a) an XGBoost-classified risk profile, (b) an LSTM forecast of category-wise spending for the next 3 months, (c) Isolation-Forest flags for anomalous transactions, (d) a Linear-Programming-optimal budget allocation, (e) a recommended SIP amount and a tax-saving plan, all narrated in natural language by FinGPT.

F.2 Input & Output Data Contracts

Input — user financial state

```
# aria/module1_personal_finance/schemas.py

from pydantic import BaseModel, Field
from datetime import date

class Transaction(BaseModel):
    date: date
    category: str # food | rent | emi | utilities | entertainment |
                # medical | travel | shopping | other
    amount: float # positive = expense, negative = income
    merchant: str | None = None
    channel: str | None = None # upi | card | cash | netbanking

class UserFinancialInput(BaseModel):
    user_id: int
    monthly_income: float
    age: int
    dependents: int = 0
    existing_emi: float = 0
    emergency_fund: float = 0 # months of expenses covered
    city_tier: int = 1 # 1 / 2 / 3 – affects min-living-cost floor
    transactions: list[Transaction] = [] # last 12 months
    tax_regime: str = "new" # "old" | "new"
    query: str = "" # the user's natural-language question
```

Output — complete M1 response

```
class RiskProfile(BaseModel):
    label: str # Conservative | Moderate | Aggressive
    confidence: float
    top_features: list[tuple[str, float]] # (feature, SHAP-value)

class CategoryForecast(BaseModel):
    category: str
    forecast: list[float] # next 3 months, monthly
    lower_ci: list[float]
    upper_ci: list[float]

class AnomalyFlag(BaseModel):
    transaction: Transaction
    score: float
    reason: str
```

```

class BudgetAllocation(BaseModel):
    housing: float
    food: float
    transport: float
    utilities: float
    emi: float
    sip: float
    entertainment: float
    emergency_fund_add: float

class SipPlan(BaseModel):
    monthly_sip: float
    fund_split: dict[str, float] # {"equity": 0.6, "debt": 0.4}
    tax_saver_amount: float # portion via ELSS under 80C

class M1Response(BaseModel):
    risk: RiskProfile
    forecast: list[CategoryForecast]
    anomalies: list[AnomalyFlag]
    budget: BudgetAllocation
    sip_plan: SipPlan
    natural_language: str # FinGPT's narrative explanation

```

F.3 Data Strategy — Where the Training Data Comes From

This is the trickiest part of Module 1. You will not have real anonymized transaction data at BTP scale. The BTP-faithful solution: synthesize realistic data that captures Indian household financial dynamics, and be explicit about it in your thesis. This is standard practice in financial ML research where real consumer data is regulated.

Synthetic data generator — what it produces

- ~500 synthetic users across 5 personas: young urban IT professional, established mid-career family, early-career graduate, near-retirement conservative saver, self-employed variable income.
- Each user: demographic attributes + 12-18 months of daily transactions + ground-truth risk label + category-wise income/expense splits.
- Transaction structure: salary credit on a fixed day, monthly rent+EMI on fixed days, variable daily spends drawn from per-category lognormal distributions, festive spikes around Diwali/Dussehra, quarterly insurance/SIP debits.

Why this is defensible

The BTP report §3.4.1 states: "Initial training uses curated synthetic data representative of Indian household financial profiles." This is exactly that — you're implementing what the report specifies.

```

# scripts/generate_synthetic_transactions.py - outline

PERSONAS = {
    "young_it_pro": dict(
        income=(90_000, 160_000), age=(24, 30), dependents=0,
        rent_ratio=(0.25, 0.35), emi_ratio=(0, 0.15),
        risk_label="Aggressive",
    ),
    "midcareer_family": dict(
        income=(150_000, 280_000), age=(34, 45), dependents=(1, 3),
        rent_ratio=(0.20, 0.30), emi_ratio=(0.25, 0.40),
        risk_label="Moderate",
    ),
}

```

```

    ),
    "early_grad": dict(
        income=(30_000, 65_000), age=(22, 26), dependents=0,
        rent_ratio=(0.30, 0.45), emi_ratio=(0, 0.10),
        risk_label="Moderate",
    ),
    "near_retirement": dict(
        income=(80_000, 180_000), age=(52, 60), dependents=(0, 2),
        rent_ratio=(0, 0.15), emi_ratio=(0, 0.10),
        risk_label="Conservative",
    ),
    "self_employed": dict(
        income=(60_000, 200_000), income_volatility=0.35,
        age=(28, 50), dependents=(0, 3),
        risk_label="Moderate",
    ),
)
}

CATEGORY_LOGNORMAL = {
    # category : (mu, sigma) of ln(daily-spend) relative to income
    "food": (-4.1, 0.6),
    "utilities": (-5.8, 0.3),
    "entertainment": (-5.0, 0.9),
    "shopping": (-4.6, 1.1),
    "medical": (-5.5, 1.4),
    "travel": (-5.1, 1.3),
    "other": (-5.3, 1.0),
}

FESTIVE_MULTIPLIERS = { # month-based multiplier on shopping & food
    # Indian calendar approximations
    10: 1.8, # October – Dussehra
    11: 2.2, # November – Diwali
    3: 1.3, # March – Holi
    12: 1.2, # December – year-end
}

def generate_user(persona_name, n_months=15): ...
def main():
    for pname in PERSONAS:
        for i in range(100): # 100 users per persona = 500 total
            user = generate_user(pname)
            save_to(f"data/raw/transactions_synthetic/{pname}_{i:03d}.parquet")

```

F.4 Component F.1 — XGBoost Risk Profiler

Training features (20 total, all derived per user)

Feature	Derivation
income_log	log(monthly_income)
savings_rate	1 – total_monthly_spend / income
debt_to_income	existing_emi / income
age	Direct
dependents	Direct

emergency_fund_months	Direct
discretionary_ratio	(entertainment + shopping + travel) / income
essential_ratio	(food + rent + utilities) / income
income_volatility	σ of monthly income over 12 months / mean
spend_volatility	σ of monthly spend over 12 months / mean
max_single_txn_ratio	max(transaction) / monthly_income
city_tier	1 / 2 / 3 one-hot
pct_cash	cash transactions / total count
pct_upi	UPI transactions / total count
avg_monthly_categories	# of distinct categories used per month
festive_spike	max(monthly spend) / median(monthly spend)
neg_balance_months	# of months where spend > income
rent_ratio	rent / income
medical_ratio	medical spend / income
log_txn_count	log(total transactions in window)

Training script essentials

```
# aria/module1_personal_finance/train/train_xgb_risk.py

import pandas as pd, xgboost as xgb, shap, joblib
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.preprocessing import LabelEncoder
from aria.module1_personal_finance.features import build_user_feature_row
from config.paths import RAW_TRANSACTIONS_DIR, XGB_RISK_PATH

def load_all_users():
    rows = []
    for path in RAW_TRANSACTIONS_DIR.glob("*.parquet"):
        df = pd.read_parquet(path)
        meta = df.attrs # persona metadata stored here at generation time
        rows.append(**build_user_feature_row(df, meta), "label": meta["risk_label"])
    return pd.DataFrame(rows)

def main():
    df = load_all_users()
    le = LabelEncoder(); y = le.fit_transform(df["label"])
    X = df.drop(columns=["label"])

    clf = xgb.XGBClassifier(
        objective="multi:softprob", num_class=3,
        eval_metric="mlogloss", tree_method="hist",
        device="cuda", # uses GPU if built with it, falls back silently
    )
    params = {
```

```

    "n_estimators": [200, 400, 800],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.03, 0.1],
    "min_child_weight": [1, 3],
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grid = GridSearchCV(clf, params, cv=cv, scoring="f1_macro", n_jobs=-1).fit(X, y)

print("best score:", grid.best_score_, "best params:", grid.best_params_)

# explainability – SHAP values for top features
explainer = shap.TreeExplainer(grid.best_estimator_)

grid.best_estimator_.save_model(XGB_RISK_PATH)
joblib.dump({"encoder": le, "explainer": explainer}, XGB_RISK_PATH + ".artifacts")

```

Why XGBoost, not a neural network (BTP §3.4.1 justification made concrete)

- Tabular features, ~500 training rows — far below the regime where neural networks beat boosted trees.
- SHAP values give per-user feature importance, enabling the FinGPT explanation to say "your profile is Moderate primarily because your debt-to-income is 0.35" — a neural network's attributions are noisier.
- GridSearchCV + 5-fold training completes in under 5 minutes on CPU; the cost-benefit of a neural model is negative here.

F.5 Component F.2 — LSTM Spending Forecast

Problem framing

Input: user's last 12 months of aggregated category-wise spends (a 12×8 tensor — 12 months $\times$ 8 categories) + contextual features per month (month-of-year one-hot, salary-day indicator). Output: next 3 months $\times$ 8 categories = 24 values, plus a quantile (10/50/90) for confidence intervals.

Architecture — matching BTP §3.4.2

```

# aria/module1_personal_finance/spend_forecast.py

import torch, torch.nn as nn

N_CATS, N_CTX, H, L, HORIZON = 8, 12, 128, 2, 3
N_FEATURES = N_CATS + N_CTX

class SpendLSTM(nn.Module):
    """Stacked LSTM with quantile regression heads."""
    def __init__(self, n_features=N_FEATURES, hidden=H, layers=L,
                  horizon=HORIZON, n_categories=N_CATS):
        super().__init__()
        self.lstm = nn.LSTM(n_features, hidden, num_layers=layers,
                             batch_first=True, dropout=0.3)
        # three quantile heads – 0.1, 0.5, 0.9
        self.head_q10 = nn.Linear(hidden, horizon * n_categories)
        self.head_q50 = nn.Linear(hidden, horizon * n_categories)
        self.head_q90 = nn.Linear(hidden, horizon * n_categories)
        self.horizon, self.n_cats = horizon, n_categories

    def forward(self, x):
        # x : (B, T=12, n_features)

```

```

        out, _ = self.lstm(x)
        h = out[:, -1, :] # last timestep
        def shape(y): return y.view(-1, self.horizon, self.n_cats)
        return shape(self.head_q10(h)), shape(self.head_q50(h)), shape(self.head_q90(h))

def quantile_loss(pred, target, q):
    diff = target - pred
    return torch.maximum(q * diff, (q - 1) * diff).mean()

def total_loss(q10, q50, q90, y):
    return (quantile_loss(q10, y, 0.1)
            + quantile_loss(q50, y, 0.5)
            + quantile_loss(q90, y, 0.9))

```

Training dataset construction

For each synthetic user with 15 months of data, create sliding windows: months 1-12 → predict 13-15, then months 2-13 → predict 14-15 (truncated), etc. 500 users × ~3 windows ≈ 1500 training samples. Small, but LSTM generalizes because the pattern (festivals, category seasonality) is consistent across personas.

Why LSTM, not ARIMA / Prophet (BTP §3.4.2 justification)

- Multivariate: 8 correlated category series + context features. ARIMA is univariate.
- Non-linear cross-category effects: a salary-increase month shifts food AND shopping AND entertainment together. LSTM captures this via shared hidden state.
- Prophet is per-series and assumes additive seasonality — good for one aggregate series but weaker for 8 joint series with the persona-conditioning the BTP report emphasizes.

F.6 Component F.3 — Isolation Forest Fraud/Anomaly Detection

Features per transaction

- amount / user's median amount in that category (scale-free)
- log(amount)
- hour of day (0-23)
- day of week (0-6)
- transactions_in_last_hour (velocity feature)
- is_new_merchant flag
- deviation_from_category_mean (z-score in that user's category)

```

# aria/module1_personal_finance/fraud_detect.py

import numpy as np, joblib
from sklearn.ensemble import IsolationForest
from aria.module1_personal_finance.schemas import Transaction, AnomalyFlag

class FraudDetector:
    def __init__(self, contamination: float = 0.02):
        self.model = IsolationForest(
            n_estimators=200, contamination=contamination,
            random_state=42, n_jobs=-1,
        )
        self.fitted = False

```

```

def fit(self, user_txn_df):
    X = self._build_features(user_txn_df)
    self.model.fit(X)
    self.fitted = True

def detect(self, txns: list[Transaction]) -> list[AnomalyFlag]:
    import pandas as pd
    df = pd.DataFrame([t.model_dump() for t in txns])
    X = self._build_features(df)
    scores = -self.model.score_samples(X)    # higher = more anomalous
    preds = self.model.predict(X)           # -1 = anomaly
    flags = []
    for i, t in enumerate(txns):
        if preds[i] == -1:
            flags.append(AnomalyFlag(
                transaction=t, score=float(scores[i]),
                reason=self._reason(t, df.iloc[i]),
            ))
    return flags

def _reason(self, t, feat_row):
    if feat_row["amount_vs_median"] > 10:
        return f"Amount {t.amount:.0f} is >10x median for category {t.category}."
    if feat_row["hour"] < 5 or feat_row["hour"] > 23:
        return f"Unusual hour ({int(feat_row['hour']):02d}:00) for this user."
    return "Multiple weak anomaly signals combined."

```

Important nuance: Isolation Forest is trained per user. One global IF across all users misses the key signal — "this is anomalous for THIS user," not "this is anomalous in general."

F.7 Component F.4 — Linear Programming Budget Optimizer

LP formulation

```

# aria/module1_personal_finance/budget_optimize.py

import numpy as np
from scipy.optimize import linprog
from aria.module1_personal_finance.schemas import BudgetAllocation, RiskProfile

# Decision variables (₹ per month):
# x = [housing, food, transport, utilities, emi, sip, entertainment, emergency]
#
# Objective: MAXIMIZE sip (savings-toward-growth). linprog minimizes, so flip sign.
#
# Equality constraint: sum(x) = income
# Inequalities:
# housing    <= 0.35 * income
# emi        == existing_emi          (hard-fixed)
# emergency  >= 0.05 * income          (unless fund already > 6 months)
# food       >= min_food_floor[city_tier]
# utilities  >= min_utilities_floor[city_tier]
# entertainment <= 0.15 * income
# risk-tier modifier: sip <= max_sip_ratio[risk_class] * income

FLOORS = {
    # minimum monthly by city tier
    1: dict(food=10000, utilities=4000, transport=3000),
    2: dict(food=7500,  utilities=3000, transport=2500),
    3: dict(food=5000,  utilities=2000, transport=2000),
}

```

```

SIP_CAP = {"Conservative": 0.15, "Moderate": 0.25, "Aggressive": 0.35}

def optimize_budget(income, existing_emi, city_tier, risk: RiskProfile,
                    emergency_months_covered: float) -> BudgetAllocation:
    # minimize -sip == maximize sip
    c = np.zeros(8); c[5] = -1.0

    A_eq = np.ones((1, 8)); b_eq = np.array([income])
    # EMI = existing_emi (hard constraint as additional equality)
    A_eq = np.vstack([A_eq, np.eye(8)[4:5]]); b_eq = np.append(b_eq, existing_emi)

    floor = FLOORS[city_tier]
    # bounds: lower floors + hard upper caps
    emergency_floor = 0.05 * income if emergency_months_covered < 6 else 0
    sip_max = SIP_CAP[risk.label] * income
    bounds = [
        (0, 0.35 * income), # housing
        (floor["food"], None),
        (floor["transport"], None),
        (floor["utilities"], None),
        (existing_emi, existing_emi), # EMI fixed
        (0, sip_max), # SIP bounded by risk class
        (0, 0.15 * income), # entertainment cap
        (emergency_floor, None),
    ]

    res = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError(f"LP infeasible: {res.message}")
    x = res.x
    return BudgetAllocation(
        housing=round(x[0], 0), food=round(x[1], 0),
        transport=round(x[2], 0), utilities=round(x[3], 0),
        emi=round(x[4], 0), sip=round(x[5], 0),
        entertainment=round(x[6], 0),
        emergency_fund_add=round(x[7], 0),
    )

```

The 50-30-20 rule is a starting point, not the answer. LP handles the cases where 50-30-20 is infeasible — e.g., someone with a 45% EMI obligation where fixed costs alone exceed 50%.

F.8 Component F.5 — SIP / EMI Calculators & Tax Rules

These are deterministic closed-form functions — no ML. Standard finance formulas, but putting them in the codebase means FinGPT explanations are grounded in real numbers rather than hallucinated ones.

```

# aria/module1_personal_finance/sip_emi_calc.py

def sip_future_value(monthly: float, years: float, annual_return_pct: float) -> float:
    """FV of monthly SIP with end-of-period contribution."""
    r = annual_return_pct / 12 / 100
    n = int(years * 12)
    return monthly * ((pow(1 + r, n) - 1) / r) * (1 + r)

def sip_required(target: float, years: float, annual_return_pct: float) -> float:
    r = annual_return_pct / 12 / 100

```

```

    n = int(years * 12)
    return target / (((pow(1 + r, n) - 1) / r) * (1 + r))

def emi(principal: float, annual_rate_pct: float, years: float) -> float:
    r = annual_rate_pct / 12 / 100
    n = int(years * 12)
    return principal * r * pow(1 + r, n) / (pow(1 + r, n) - 1)

```

Tax rules (new regime — FY 2025-26 slabs)

```

# aria/module1_personal_finance/tax_rules.py

NEW_REGIME_SLABS_FY25_26 = [
    (300_000, 0.00), # up to 3L — nil
    (700_000, 0.05),
    (1_000_000, 0.10),
    (1_200_000, 0.15),
    (1_500_000, 0.20),
    (float("inf"), 0.30),
]

STD_DEDUCTION = 75_000 # standard deduction, new regime, FY25-26

OLD_REGIME_80C_CAP = 150_000
OLD_REGIME_80D_CAP_SELF = 25_000
OLD_REGIME_80D_CAP_SENIOR = 50_000

def tax_new(gross_income: float) -> float:
    taxable = max(0, gross_income - STD_DEDUCTION)
    tax, prev = 0.0, 0.0
    for slab, rate in NEW_REGIME_SLABS_FY25_26:
        if taxable <= slab:
            tax += (taxable - prev) * rate
            break
        tax += (slab - prev) * rate
        prev = slab
    return tax * 1.04 # +4% cess

```

Tax law changes every budget cycle. Keep `tax_rules.py` hygienic — one module, slab constants clearly dated. This is what you want a tax CA to audit, not what FinGPT generates freeform.

F.9 Module 1 Orchestrator

```

# aria/module1_personal_finance/orchestrator.py

from aria.module1_personal_finance.schemas import UserFinancialInput, M1Response
from aria.module1_personal_finance import risk_profile, spend_forecast, fraud_detect
from aria.module1_personal_finance import budget_optimize, sip_emi_calc, tax_rules
from aria.shared import fingpt_client, taxal

def run(inp: UserFinancialInput) -> M1Response:
    # 1. ML predictions
    risk = risk_profile.predict(inp)
    forecast = spend_forecast.predict(inp)
    flags = fraud_detect.detect_for_user(inp)
    budget = budget_optimize.optimize_budget(
        inp.monthly_income, inp.existing_emi, inp.city_tier,
        risk, inp.emergency_fund)

```

```

# 2. Derived products
sip_plan = _build_sip_plan(budget.sip, risk, inp.tax_regime, inp.monthly_income)

# 3. FinGPT narration – grounded in the computed numbers
prompt = _build_narration_prompt(inp, risk, forecast, flags, budget, sip_plan)
narration = fmgpt_client.generate(prompt, max_new_tokens=500, temperature=0.3).text

return M1Response(risk=risk, forecast=forecast, anomalies=flags,
                  budget=budget, sip_plan=sip_plan, natural_language=narration)

def _build_sip_plan(sip_amount, risk, tax_regime, income):
    equity_frac = {"Conservative": 0.3, "Moderate": 0.6, "Aggressive": 0.8}[risk.label]
    # ELSS cap under 80C = ₹12,500/month (1.5L/12). Only in old regime.
    tax_saver_cap = 12_500 if tax_regime == "old" else 0
    tax_saver = min(sip_amount * equity_frac, tax_saver_cap)
    return SipPlan(
        monthly_sip=sip_amount,
        fund_split={"equity": equity_frac, "debt": 1 - equity_frac},
        tax_saver_amount=tax_saver,
    )

NARRATION_PROMPT = """You are FinGPT, an AI financial advisor. Below is a computed
financial analysis for one user. Write a 4-6 sentence personalised narrative for the user.
Use ONLY the numbers provided. Do not invent any figures. Use Indian context.

User profile:
Monthly income:    ₹{income:,.0f}
Age:               {age}
City tier:         {city}
Existing EMI:      ₹{emi:,.0f}

Computed outputs:
Risk class:        {risk_label} (confidence {risk_conf:.2f})
Recommended SIP:   ₹{sip:,.0f}/month
Equity/debt split: {equity_pct:.0%} / {debt_pct:.0%}
Emergency fund top-up: ₹{ef:,.0f}
Next-month spending forecast (total): ₹{forecast_total:,.0f}

Narrative:"""

```

F.10 Training Pipeline — Module 1

Run these in order once, before the first demo.

```

# 1. Generate synthetic training data (500 users, ~15 min on CPU)
python -m scripts.generate_synthetic_transactions

# 2. Train XGBoost risk classifier (~5 min)
python -m aria.module1_personal_finance.train.train_xgb_risk

# 3. Train LSTM spending forecaster (~30 min on GPU)
python -m aria.module1_personal_finance.train.train_lstm_spend

# 4. Fit a generic Isolation Forest (~2 min) – per-user fitting happens at query time

```

```
python -m aria.module1_personal_finance.train.train_iso_forest
```

```
# 5. Smoke-test the orchestrator end-to-end
```

```
python -m aria.module1_personal_finance.orchestrator --demo
```

Part G — Module 2: Intelligent Financial Tutor

G.1 What Module 2 Does — In One Paragraph

Given a user's free-text question about a financial concept and their per-concept mastery history, Module 2: (a) retrieves the most relevant concept from a curated 50-concept knowledge base using Sentence-BERT, (b) estimates current mastery with an LSTM Knowledge Tracing model, (c) chooses the optimal teaching action (simplify / give example / quiz / deepen / analogy / prerequisite) with a trained DQN agent, (d) generates a TAXAL-structured explanation at the appropriate depth level using FinGPT, and (e) updates mastery state based on the user's next interaction.

G.2 Concept Knowledge Base

The BTP report mentions a KB of ~2000 concepts for production. For the demo, 50 well-chosen concepts cover enough breadth for adaptive-tutoring to demonstrate its value. Each entry:

```
# data/raw/concepts_kb/concepts.json — one entry format

{
  "id": "compound_interest",
  "canonical_name": "Compound Interest",
  "aliases": ["compounding", "interest on interest", "power of compounding"],
  "category": "time_value_of_money",
  "prerequisites": ["simple_interest", "percentage_basics"],
  "difficulty_floor": 1,      # minimum TAXAL level this can be taught at
  "difficulty_ceiling": 8,   # max meaningful depth
  "keywords": ["CAGR", "XIRR", "FV", "time value"],
  "seed_definition":
    "Compound interest is interest calculated on both the initial principal
    and the accumulated interest from previous periods.",
  "related_concepts": ["cagr", "xirr", "future_value", "inflation_adjusted_return"]
}
```

Suggested 50-concept coverage (to hand-author)

Category	Concepts (sample)
Time value of money	simple interest, compound interest, CAGR, XIRR, future value, present value, NPV
Investment vehicles	mutual fund, SIP, ELSS, ETF, index fund, FD, PPF, NPS, direct equity
Risk concepts	volatility, standard deviation, beta, sharpe ratio, drawdown, diversification
Market mechanics	stock exchange, settlement cycle, bid-ask, market cap, P/E, dividend yield
Taxation (India)	80C, 80D, LTCG, STCG, new vs old regime, HRA, standard deduction
Personal finance	emergency fund, debt trap, credit score, net worth, inflation, real return

Loans	EMI, interest rate types (fixed/floating), amortization, pre-payment, CIBIL score
Retirement planning	corpus target, inflation-adjusted need, annuity, withdrawal rate

```
# scripts/build_concept_kb.py

from aria.shared import vector_store
import json, pathlib

KB = json.loads(pathlib.Path("data/raw/concepts_kb/concepts.json").read_text())

# Build text-for-embedding from each concept – combine name + aliases + seed_def
texts, meta = [], []
for c in KB:
    blob = f"{c['canonical_name']}. {' '.join(c['aliases'])}. {c['seed_definition']}"
    texts.append(blob)
    meta.append(c) # full concept dict saved so retriever returns rich objects

vector_store.build_index(namespace="concepts", texts=texts, metadata=meta)
```

G.3 Concept Retriever

```
# aria/module2_tutor/concept_retriever.py

from aria.shared import vector_store

def retrieve(query: str, k: int = 3) -> list[dict]:
    hits = vector_store.search("concepts", query, k=k)
    # Accept only if top hit has score > 0.45 (cosine on normalized SBERT vectors).
    # Below that, caller treats as 'no confident match – ask for clarification'.
    return hits

def best(query: str, threshold: float = 0.45) -> dict | None:
    hits = retrieve(query, k=1)
    if hits and hits[0]["score"] >= threshold:
        return hits[0]
    return None
```

G.4 LSTM Knowledge Tracing — Deep Knowledge Tracing (DKT)

The BTP report §4.6.1 specifies LSTM-based Knowledge Tracing with one-hot 2K-dimensional input (K = number of concepts, positions 0..K-1 mark correct answer, K..2K-1 mark incorrect). This is the canonical Piech et al. 2015 DKT architecture. K=50 → 100-dim input. Well within demo scope.

```
# aria/module2_tutor/knowledge_tracer.py

import torch, torch.nn as nn

class DKT(nn.Module):
    """Deep Knowledge Tracing (Piech et al., 2015).
    Input : (B, T, 2K) – one-hot over (concept_id, correctness).
    Output : (B, T, K) – probability the user would answer each concept correctly at t+1.
    """
    def __init__(self, n_concepts: int, hidden: int = 128, layers: int = 2):
        super().__init__()
        self.n = n_concepts
```

```

self.lstm = nn.LSTM(2 * n_concepts, hidden, num_layers=layers,
                    batch_first=True, dropout=0.2)
self.head = nn.Linear(hidden, n_concepts)

def forward(self, x):
    h, _ = self.lstm(x)
    return torch.sigmoid(self.head(h))

def encode_interaction(concept_ids: list[int], correct: list[int], n_concepts: int):
    T = len(concept_ids)
    x = torch.zeros(T, 2 * n_concepts)
    for t, (c, y) in enumerate(zip(concept_ids, correct)):
        idx = c if y == 1 else c + n_concepts
        x[t, idx] = 1.0
    return x

def mastery_vector(model: DKT, history) -> dict:
    """Current mastery probability for every concept."""
    if not history:
        return {i: 0.0 for i in range(model.n)}
    c_ids = [h["concept_id"] for h in history]
    corrects = [h["is_correct"] for h in history]
    x = encode_interaction(c_ids, corrects, model.n).unsqueeze(0)
    with torch.no_grad():
        probs = model(x)[0, -1, :] # last timestep
    return {i: float(probs[i]) for i in range(model.n)}

```

Training data — where it comes from

Problem: we have no real student-interaction data. Solution — simulate it from a plausible model of learner behavior (the standard approach in DKT research for cold-start evaluation):

4. Define 500 synthetic learners with latent ability $a \in N(0, 1)$.
5. Define each concept with latent difficulty $d \in N(0, 1)$ and use Item Response Theory $P(\text{correct}) = \sigma(a - d + \text{noise})$.
6. Simulate 40-80 interaction-sequences per learner respecting prerequisites (can't answer CAGR correctly without simple_interest mastery).
7. Train DKT on these sequences. Expected AUC on held-out sequences: 0.80-0.85 — sufficient for a BTP demo and matches published DKT numbers.

Disclose this clearly in your thesis: "DKT is trained on simulated IRT-based interactions, standard in Knowledge Tracing research for bootstrapping; the architecture and learned representations are genuine — only the training corpus is synthetic." This is an honest and defensible framing.

G.5 DQN Teaching Agent

RL formulation — one table, no ambiguity

Element	Definition
---------	------------

State	[mastery_target (scalar), avg_mastery_prereqs (scalar), last_5_accuracy (scalar), session_length (scalar), concept_difficulty (scalar)] — a 5-dim vector, one per query
Actions	{simplify_level_down, give_example, ask_quiz, increase_level, use_analogy, teach_prerequisite} — 6 discrete actions
Reward	+2 for concept-mastery crossing 0.85 · +1 correct quiz on target · -0.5 incorrect · -0.1 per action (encourages efficient learning)
Episode	A single learner's session on one concept; terminates at mastery ≥ 0.85 OR 10 actions elapsed
Algorithm	DQN with experience replay + target network (stable-baselines3)

```
# aria/module2_tutor/teaching_agent.py

import numpy as np, gymnasium as gym
from gymnasium import spaces
from stable_baselines3 import DQN

ACTIONS = [
    "simplify_level_down", "give_example", "ask_quiz",
    "increase_level", "use_analogy", "teach_prerequisite",
]

class TutoringEnv(gym.Env):
    """Gym environment for offline DQN training.
    Uses a simulated learner (same IRT model used for DKT pretraining) to roll out
    how actions change mastery.
    """
    def __init__(self, n_concepts=50):
        super().__init__()
        self.action_space = spaces.Discrete(len(ACTIONS))
        self.observation_space = spaces.Box(low=0.0, high=1.0, shape=(5,), dtype=np.float32)
        self.n_concepts = n_concepts

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.state = self._sample_initial_state()
        self.steps = 0
        return self.state, {}

    def step(self, action):
        mastery_before = self.state[0]
        mastery_after, quiz_ok = self._transition(action)
        reward = -0.1
        if quiz_ok is True: reward += 1.0
        if quiz_ok is False: reward -= 0.5
        if mastery_after >= 0.85 and mastery_before < 0.85: reward += 2.0
        self.state = np.array([mastery_after, *self.state[1:]], dtype=np.float32)
        self.steps += 1
        terminated = mastery_after >= 0.85 or self.steps >= 10
        return self.state, reward, terminated, False, {}

    def _sample_initial_state(self):
        return np.array([
```

```

        np.random.uniform(0.0, 0.4),    # mastery_target – start low
        np.random.uniform(0.3, 0.9),    # prereq avg
        np.random.uniform(0.3, 0.7),    # last-5 accuracy
        0.0,                             # session length
        np.random.uniform(0.2, 0.8),    # concept difficulty
    ], dtype=np.float32)

    def _transition(self, action):
        # Simple learner model: each action produces a mastery delta conditioned on state.
        # Calibrate these constants so the training data matches real-student gain
        patterns.
        base_gain = {0: 0.05, 1: 0.08, 2: 0.10, 3: 0.03, 4: 0.07, 5: 0.06}[action]
        prereq_bonus = 0.05 * (self.state[1] - 0.5)
        gain = base_gain + prereq_bonus + np.random.normal(0, 0.02)
        new_mastery = np.clip(self.state[0] + gain, 0, 1)
        quiz_ok = None
        if action == 2:    # quiz action has a pass/fail outcome
            quiz_ok = np.random.random() < new_mastery
        return new_mastery, quiz_ok

def train(total_timesteps: int = 200_000, save_path: str = "models/dqn_teacher.zip"):
    env = TutoringEnv()
    model = DQN(
        "MlpPolicy", env,
        learning_rate=3e-4, buffer_size=50_000,
        batch_size=64, train_freq=4, target_update_interval=1000,
        exploration_fraction=0.3, verbose=1,
    )
    model.learn(total_timesteps=total_timesteps)
    model.save(save_path)

```

Why DQN and not a bandit / rule-based system (BTP §4.6.2)

- Teaching is sequential, not one-shot. The optimal action at step 3 depends on what happened at steps 1-2. A bandit treats each step independently.
- Reward is delayed: choosing "teach_prerequisite" hurts immediate quiz score but unlocks the target concept later. Q-learning propagates this delayed reward; a bandit cannot.
- DQN handles continuous-valued state (mastery probabilities) better than tabular Q-learning without discretization loss.

G.6 TAXAL Explainer (Module-2 Specifics)

G.6 uses `aria/shared/taxal.py` (Part E.7) as-is. The only module-2-specific piece is the prompt enrichment — injecting the retrieved concept's prerequisites and aliases so FinGPT has context:

```

# aria/module2_tutor/explainer.py

from aria.shared import taxal, fmgpt_client

def explain_with_context(concept_record: dict, level: int,
                        teaching_action: str) -> taxal.TaxalExplanation:
    """Wraps taxal.explain with concept-specific grounding and DQN-chosen style."""
    STYLE_HINTS = {
        "simplify_level_down": "Use very simple language. Use an everyday analogy.",
        "give_example":        "Include a concrete numerical example.",
        "use_analogy":         "Lead with a single memorable analogy before definition.",
        "teach_prerequisite":   "Start by explaining a prerequisite concept, then connect.",
    }

```

```

        "increase_level":      "Include a formula and derivation.",
        "ask_quiz":           "End with one concrete quiz question the user should
answer.",
    }
    extra = STYLE_HINTS.get(teaching_action, "")
    # build enriched prompt on top of taxal's standard prompt
    prompt = taxal.PROMPT.format(
        concept=concept_record["canonical_name"], level=level,
        hint=taxal.LEVEL_HINTS[level] + " " + extra,
        length_hint=taxal._LENGTH[level],
    )
    if concept_record.get("prerequisites"):
        prompt += f"\nRelevant prerequisites to assume: {'',
'.join(concept_record['prerequisites'])}.'"
    r = fingpt_client.generate(prompt, max_new_tokens=500, temperature=0.3)
    return taxal._parse(r.text, level)

```

G.7 Quiz Engine

```

# aria/module2_tutor/quiz_engine.py

from aria.shared import fingpt_client
from dataclasses import dataclass

@dataclass
class QuizItem:
    question:      str
    options:       list[str]      # 4 options, first is correct (shuffled by caller)
    explanation:   str
    concept_id:    str

QUIZ_PROMPT = """Generate one multiple-choice question about "{concept}" at difficulty level
{level}/8.

Format EXACTLY as follows, no additional text:

QUESTION: <one clear question>
CORRECT: <the correct answer, 1-2 sentences>
WRONG_1: <a plausible but incorrect answer>
WRONG_2: <another plausible but incorrect answer>
WRONG_3: <another plausible but incorrect answer>
EXPLANATION: <one sentence explaining why CORRECT is right>
"""

def generate_quiz(concept_record, level: int) -> QuizItem:
    prompt = QUIZ_PROMPT.format(concept=concept_record["canonical_name"], level=level)
    text = fingpt_client.generate(prompt, max_new_tokens=400, temperature=0.4).text
    return _parse_quiz(text, concept_record)

```

G.8 Module 2 Orchestrator

```

# aria/module2_tutor/orchestrator.py

from stable_baselines3 import DQN
import numpy as np, torch
from aria.module2_tutor import concept_retriever, explainer, quiz_engine
from aria.module2_tutor.knowledge_tracer import DKT, mastery_vector
from aria.module2_tutor.teaching_agent import ACTIONS

```

```

from aria.shared import user_store
from config.paths import LSTM_KT_PATH, DQN_TEACHER_PATH

_kt_model = None; _dqn_model = None
def _load_models(n_concepts):
    global _kt_model, _dqn_model
    if _kt_model is None:
        _kt_model = DKT(n_concepts); _kt_model.load_state_dict(torch.load(LSTM_KT_PATH))
        _kt_model.eval()
    if _dqn_model is None:
        _dqn_model = DQN.load(DQN_TEACHER_PATH)

def handle(user_id: int, query: str, n_concepts: int = 50):
    _load_models(n_concepts)

    # 1. Retrieve concept
    concept = concept_retriever.best(query)
    if concept is None:
        return {"reply": "I couldn't match that to a concept I know. Could you rephrase?"}

    # 2. Mastery estimation from user's history
    history = user_store.get_learning_history(user_id)
    masteries = mastery_vector(_kt_model, history)
    target_mastery = masteries[concept["index"]]
    prereq_masteries = [masteries[p] for p in concept.get("prereq_indices", [])]
    prereq_avg = float(np.mean(prereq_masteries)) if prereq_masteries else 0.5

    # 3. Build state, ask DQN for action
    state = np.array([target_mastery, prereq_avg, _last5_acc(history),
                     0.0, concept["difficulty_norm"]], dtype=np.float32)
    action_idx, _ = _dqn_model.predict(state, deterministic=True)
    action = ACTIONS[int(action_idx)]

    # 4. Map mastery → TAXAL level
    level = _mastery_to_level(target_mastery)

    # 5. Generate TAXAL explanation
    taxal_exp = explainer.explain_with_context(concept, level, action)

    # 6. If DQN chose ask_quiz, append a quiz
    quiz = quiz_engine.generate_quiz(concept, level) if action == "ask_quiz" else None

    return {
        "concept": concept["canonical_name"],
        "level": level,
        "action": action,
        "mastery": target_mastery,
        "taxal": taxal_exp,
        "quiz": quiz,
    }

def _mastery_to_level(m: float) -> int:
    # mastery 0.0 → level 1; mastery 1.0 → level 6; above that the user asks for depth explicitly
    return int(1 + m * 5) if m < 1.0 else 6

def _last5_acc(history):

```

```
if not history: return 0.5
last = history[-5:]
return sum(h["is_correct"] for h in last) / len(last)
```

G.9 Training Pipeline — Module 2

```
# 1. Hand-author the 50-concept KB (manual; the hardest single task in M2)
#    - use scripts/seed_concepts.py as a skeleton

# 2. Build the FAISS index (~30 seconds)
python -m scripts.build_concept_kb

# 3. Generate synthetic KT interaction sequences (~10 min)
python -m scripts.generate_kt_sequences

# 4. Train LSTM-KT model (~45 min on GPU for 500 learners x 50 epochs)
python -m aria.module2_tutor.train.train_lstm_kt

# 5. Train DQN teaching agent (~60 min on CPU for 200k timesteps)
python -m aria.module2_tutor.train.train_dqn_teacher

# 6. Smoke test
python -m aria.module2_tutor.orchestrator --demo
```

Part H — Integration & CLI Demo

H.1 The Router

```
# aria/shared/router.py

from aria.shared import intent_classifier, ner_extractor
from aria.module1_personal_finance import orchestrator as m1
from aria.module2_tutor import orchestrator as m2

M1_INTENTS = {"budget_plan", "sip_advice", "emi_compare", "tax_plan",
              "expense_forecast", "fraud_check", "risk_profile"}
M2_INTENTS = {"explain_concept", "quiz_me", "learning_path"}

def route(user_id: int, query: str, user_context: dict):
    intent, conf = intent_classifier.classify(query)
    if conf < 0.5:
        return {"reply": "I'm not sure what you're asking. Could you rephrase?", "intent":
intent}
    if intent in M1_INTENTS:
        m1_input = _build_m1_input(user_id, query, intent, user_context)
        return {"module": "M1", "intent": intent, "response": m1.run(m1_input)}
    if intent in M2_INTENTS:
        return {"module": "M2", "intent": intent, "response": m2.handle(user_id, query)}
    # shared intents
    return _handle_shared(intent, query)
```

H.2 CLI Demo

```
# aria/interface/cli.py

import typer, json
from rich.console import Console
from rich.panel import Panel
from aria.shared import router, fmgpt_client
from aria.shared.user_store import get_or_create_user

app = typer.Typer()
console = Console()

@app.command()
def chat(user: str = "demo_user"):
    """Interactive Aria chat session."""
    if not fmgpt_client.health():
        console.print("[red]FinGPT server is not running.[/red] Start it with:\n"
            "    uvicorn aria.shared.fmgpt_server:app --host 127.0.0.1 --port
7860")
        raise typer.Exit(1)

    user_id = get_or_create_user(user).id
    console.print(Panel.fit("[bold cyan]Aria[/bold cyan] – Personal Finance Assistant &
Financial Tutor",
        subtitle="type /exit to quit"))

    while True:
        q = console.input("[bold green]you > [/bold green]").strip()
        if q.lower() in {"/exit", "/quit"}: break
```

```

        if not q: continue

        result = router.route(user_id, q, user_context={})
        _render(result)

def _render(result):
    if "reply" in result:
        console.print(Panel(result["reply"], title="Aria", border_style="cyan"))
        return
    mod = result.get("module")
    if mod == "M1":
        r = result["response"]
        console.print(Panel(r.natural_language, title="Aria – Finance",
border_style="cyan"))
        console.print(f" Risk: {r.risk.label} ({r.risk.confidence:.2f})")
        console.print(f" SIP: ₹{r.sip_plan.monthly_sip:,.0f}/month")
    elif mod == "M2":
        r = result["response"]
        t = r["taxal"]
        console.print(Panel(
            f"[bold]Cognitive:[/bold] {t.cognitive}\n\n"
            f"[bold]Functional:[/bold] {t.functional}\n\n"
            f"[bold]Causal:[/bold] {t.causal}",
            title=f"Aria – Tutor ({r['concept']}, level {r['level']}), action
{r['action']})",
            border_style="cyan",
        ))
        if r["quiz"]:
            console.print(Panel(r["quiz"].question, title="Quick check"))

if __name__ == "__main__":
    app()

```

H.3 Running the Demo

```

# Terminal 1 – FinGPT server (keep running)
uvicorn aria.shared.fingpt_server:app --host 127.0.0.1 --port 7860

# Terminal 2 – Aria CLI
python -m aria.interface.cli chat --user demo_user

# Example exchanges:
# you > I earn 95k/month, pay 25k rent, 8.5k EMI. Should I start a SIP?
# you > What is compound interest?
# you > Quiz me on mutual funds.
# you > How much tax do I pay on 12 lakhs under new regime?

```

Part I — Roadmap & Testing

I.1 Six-Sprint Plan

Sprint	Theme	Deliverables
1 (week 1)	Foundation	Repo skeleton · pyproject.toml · config · FinGPT server running locally · health endpoint returning OK · basic CLI shell (no models yet)
2 (week 2)	Shared layer	DistilBERT intent classifier trained & tested · DeBERTa-v3 NER working on 200 hand-labeled examples · SBERT vector store built on 50-concept KB · TAXAL renderer producing 3-section output
3 (week 3)	Module 1 ML	Synthetic transactions generated · XGBoost risk profiler (5-fold CV F1 > 0.75) · LSTM spend forecaster (MAE < 15% of income) · Isolation Forest integrated
4 (week 4)	Module 1 integration	LP budget optimizer · SIP/EMI calculators · tax_rules · full M1 orchestrator → FinGPT narration · 10 scripted end-to-end M1 test cases pass
5 (week 5)	Module 2	50-concept KB hand-authored · synthetic KT sequences · DKT trained (held-out AUC > 0.80) · DQN trained · M2 orchestrator end-to-end · 10 scripted M2 cases pass
6 (week 6)	Polish & demo	CLI · Streamlit · integration tests · demo script · fallback paths validated · thesis-ready screenshots

I.2 Acceptance Criteria — What 'Done' Looks Like

Module 1

- Given a synthetic user persona, M1 produces a valid risk label + SIP recommendation + budget + narration in <10 seconds.
- XGBoost 5-fold CV macro-F1 ≥ 0.75 across Conservative/Moderate/Aggressive.
- LSTM spend forecast: P50 forecast MAE < 15% of user's monthly income on held-out synthetic users.
- Isolation Forest catches 90% of manually-injected anomalies (10× median transactions, 3am transactions, new-merchant large transactions) on a synthetic evaluation set.
- LP budget is always feasible for the 5 defined personas; infeasibility raises a clear error (not a crash).
- FinGPT narration passes the "no invented numbers" check — every rupee figure in the output appears in the structured prediction dict.

Module 2

- Given an English question about any of the 50 concepts, concept retrieval hits the correct concept in top-1 $\geq 85\%$ of the time on 100 hand-written test queries.
- DKT held-out AUC ≥ 0.80 .

- DQN policy converges: mean reward per episode at end of training > 0 (unconverged would be negative due to -0.1 step cost).
- TAXAL output always has all three non-empty sections; the parser detects and re-prompts on malformed output.
- End-to-end latency per tutor turn < 8 seconds on the target GPU.

I.3 Testing Strategy

Unit tests — what goes in tests/

- Every module in aria/ has at least one test file in tests/ covering its public functions.
- Use pytest fixtures to load small synthetic samples (a 30-day transaction slice, a 10-interaction KT sequence) — tests should run in < 30 seconds total on CPU.
- FinGPT calls in tests are mocked via a FinGPTResponse(text="canned output", ok=True) fake — never hit the server from unit tests.

Integration tests

- tests/integration/test_end_to_end.py fires 10 hand-crafted queries (5 M1, 5 M2) through the full pipeline with a real FinGPT server. Asserted: response shape, no crashes, structural validity of TAXAL sections.
- Run this before every demo, never in CI (too heavy).

Adversarial test corpus (10 queries)

8. Empty input
9. Very long input (2000 characters)
10. Mixed Hindi-English code-switching ("मुझे SIP में invest करना है")
11. Off-topic ("what's the weather in Chennai?") → should route to chitchat, not crash
12. Ambiguous ("tell me about it") → should ask for clarification
13. Intentionally misspelled concept ("compoundd innterest") — SBERT should still retrieve
14. Query with multiple intents ("what is SIP and help me plan one") → should pick one and answer
15. Numerical stress test (income = 0, negative income)
16. Concept not in KB ("what is a CDO?") → should respond "I don't cover that yet"
17. Fraud query on a user with no transactions → should respond gracefully

I.4 Failure Handling

Failure	Detection	Response
FinGPT server down	fingpt_client.health() returns False	CLI refuses to start; Streamlit shows banner with start-command
FinGPT generation timeout	60 s timeout in client	Return a template response assembled from structured prediction dict

Malformed TAXAL output	parser returns empty sections	Re-prompt once with stricter format instruction; template fallback after 2 failures
Concept retrieval low confidence	top-1 score < 0.45	Ask clarifying question: "Did you mean X or Y?"
Intent confidence < 0.5	DistilBERT softmax < 0.5	Ask user to rephrase; never silently route
LP infeasible	linprog returns success=False	Return structured error identifying which constraint is binding (e.g., "rent + EMI > 85% of income")
Missing trained model	file not found at config path	Clear error telling user which training script to run

Part J — BTP Report → Code Mapping

J.1 Traceability Matrix

For your thesis defense — every model named in the BTP report, its exact location in the codebase, and its role.

BTP report model	Module	Code location	Status
FinGPT	both	aria/shared/fingpt_server.py	Implemented (core)
BERT / DistilBERT	M1 intent	aria/shared/intent_classifier.py	Implemented
DeBERTa-v3	M1 NER	aria/shared/ner_extractor.py	Implemented (hand-labeled fine-tune)
LLaMA 3	M1 convo	—	Deferred: FinGPT covers for demo
XGBoost	M1 risk	aria/module1_personal_finance/risk_profile.py	Implemented
LSTM / GRU	M1 spend	aria/module1_personal_finance/spend_forecast.py	LSTM implemented
K-Means	M1 segments	—	Deferred: not required for demo; hook in features.py
Isolation Forest	M1 fraud	aria/module1_personal_finance/fraud_detect.py	Implemented
Linear Programming	M1 budget	aria/module1_personal_finance/budget_optimize.py	Implemented (scipy HiGHS)
Fin-R1	M2 reasoning	aria/shared/taxal.py	Approximated via structured FinGPT prompting
Sentence-BERT / MiniLM	M2 retrieval	aria/shared/vector_store.py	MiniLM variant (report-named)
FLAN-T5	M2 simplify	—	Deferred: TAXAL level

			system handles simplification
LSTM Knowledge Tracing	M2 mastery	aria/module2_tutor/knowledge_tracer.py	Implemented (Piech DKT)
DQN	M2 teaching	aria/module2_tutor/teaching_agent.py	Implemented (stable-baselines3)
Random Forest	M2 difficulty	—	Folded into mastery→level mapping; RF adds no signal at demo scale

J.2 Thesis Defense — Answers to the Obvious Questions

"Why FinGPT and not GPT-4 or Claude?"

The BTP report (§3.3.1) specifies FinGPT because it is trained on financial corpora (SEC filings, earnings calls, Indian regulatory documents) and understands Indian-specific financial vocabulary (Section 80C, ELSS, CAGR) natively. A general LLM hallucinates financial figures; FinGPT is demonstrably less prone to this. Using a paid API also contradicts the research-openness claim of the project.

"Where is Fin-R1?"

Fin-R1's role — generating step-by-step causal reasoning chains — is filled in the demo by FinGPT with a structured prompt that forces the three TAXAL sections and uses chain-of-thought conditioning. The architectural slot for Fin-R1 is preserved in aria/shared/taxal.py; replacing the client call with a Fin-R1 call is a 5-line change when a public Fin-R1 checkpoint becomes available.

"Why synthetic training data?"

Real Indian retail-banking transaction data is not openly available at BTP scale. Synthetic data generated from persona-conditioned lognormal distributions with festival multipliers and realistic EMI/rent ratios is a standard methodology in personal-finance-ML research (cf. Saptarshi et al.) and the BTP report itself (§3.4.1) explicitly states that initial training uses curated synthetic data.

"Why XGBoost for risk, not a neural network?"

Tabular data, limited samples (~500 users), and the need for SHAP-based feature attribution. XGBoost is known to outperform neural networks on tabular data under 10k samples (Shwartz-Ziv & Armon, 2022). Neural networks would also muddy the explanation layer — FinGPT's narration cites the top SHAP features directly.

J.3 Closing

This document is your single-source-of-truth for the first six weeks of development. Every file you create belongs in one of the locations specified in Part C. Every model is trained before its orchestrator is invoked. Every orchestrator is unit-tested before integration. And every substitution away from a BTP-named model is called out in the same paragraph that uses it.

The core design principle — ML predicts, DL learns over time, Transformers explain, RL personalizes — governs what each component is allowed to do. Keep it. When tempted to have FinGPT compute a number or have XGBoost write a sentence, stop and re-assign the task. That discipline is what makes this a research-grade system rather than a prompt-chain.

— End of Implementation Plan v2.0 —